

2조 오운완

RP GYM

건강 활동을 Quest와 보상으로 연결하는
헬스케어 게이미피케이션 플랫폼

팀원 - 정나영 · 최유준 · 남건우 · 황지호 · 진혜림 · 강윤석
담당 튜터 - 엄성문 튜터님

팀스파르타주식회사 내일배움캠프
단기심화 8기 최종 프로젝트



목차

01 프로젝트 개요

02 프로젝트 팀 구성 및 역할

03 프로젝트 수행 절차 및 방법

04 부하 테스트 및 트러블 슈팅

05 자체 평가 의견

문제 선정 배경 및 기획 의도

1 문제 정의

문제	원인	해결 방안
건강 관리를 지속하기 어렵다	즉각적인 보상과 재미 부족	XP · Level · Quest 등 게임 요소로 연결
데이터를 봐도 활용법을 모른다	단순 기록과 통계만 제공	달성률 · 부족 활동 계산 → 맞춤 Quest 추천
혼자 하는 관리는 동기가 부족하다	함께 수행 · 경쟁할 요소 부족	친구 · Party · 공동 Quest · Ranking

2 기획 의도

일반 서비스 : 건강 활동 → 기록 → 통계 " 관리해야 한다 "

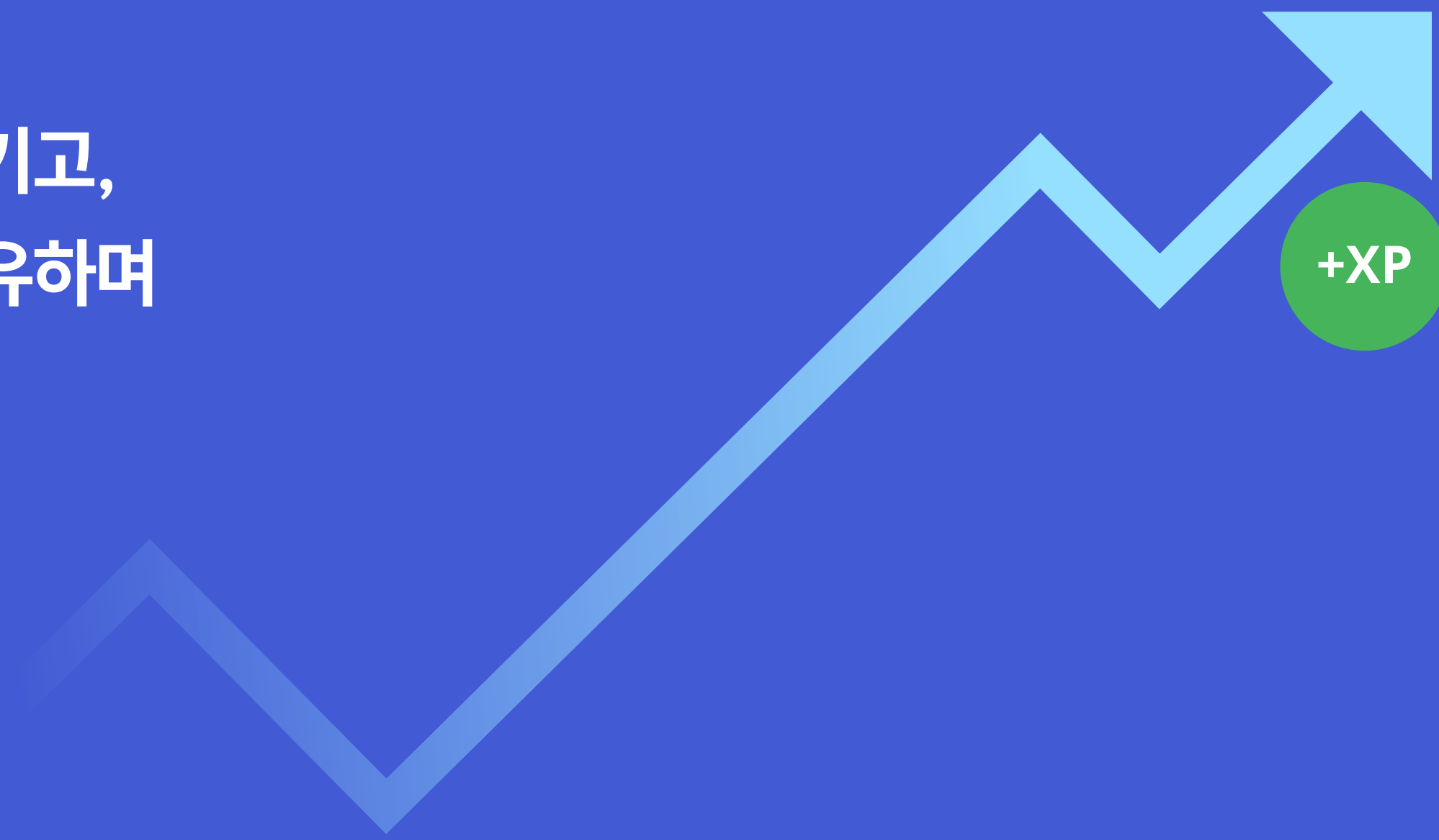
RPGGym : 수집 → 분석 → AI Quest → 활동 → 자동 완료 → XP · 성장 → Party · Ranking

" 오늘의 Quest 를 수행한다 " 는 방식으로 참여 유도

EXPECTED IMPACT

기대 효과

운동과 건강 관리를 게임처럼 즐기고,
친구들과 함께 목표와 성과를 공유하며
꾸준히 참여하도록 돕는 것



+XP

비즈니스 실무 활용성

기업 복지 - 임직원 건강 챌린지로 사용 가능 ▶

지역 커뮤니티 - 동네 주민 건강 챌린지 운영에 활용 가능 ▶

공공 건강 증진 - 국민의 일상 속 운동 참여를 유도하는 데 활용 가능 ▶

구현 사항 · 컨셉 · 훈련 내용의 연관성

컨셉 " 건강 데이터 동기화 → 달성률 분석 → AI 퀘스트 제안 → 수락 → 달성 판정 → XP · 업적 · 캐릭터 성장 " 을 이벤트 기반 MSA 로 구현

훈련 내용	프로젝트 적용 내용
애플리케이션 모니터링 (Prometheus · Grafana)	Prometheus · Grafana · Loki · Alloy · Zipkin 구성 , Outbox 발행 성공 / 실패 · 지연 · PENDING/FAILED 지표 Grafana 대시보드
시큐어 코딩	Gateway가 JWT 검증 후 X-User-Id · X-User-Role 재생성, Slack 요청 서명 검증 , actuator 노출 최소화, 요청 입력 검증 (미래 시각 · 내부 전용 소스 400), 토큰 오류 401 통일
동시성 제어 (비관적 · 낙관적 · 데드락)	낙관적 락 (version), 비관적 락 (FOR UPDATE SKIP LOCKED), 조건부 UPDATE · 부분 유니크 인덱스 , Redis Lua 원자 처리 , 정리 배치 500 건 분할로 락 점유 최소화
장애 식별 · 대응 · 후속 · 예방	etry → DLQ, 서비스 간 타임아웃 (Gateway 3s/5s · Feign), Gemini 실패 시 폴백 , Outbox 로 이벤트 유실 방지 , Blue-Green 자동 롤백 · graceful shutdown

개발 환경

Language · Framework

Java 17 Spring Boot 3.5
Spring Cloud 2025.0
(Gateway · Eureka · OpenFeign)
Spring Data JPA · Spring Retry

Data · Messaging

PostgreSQL 16 (서비스별 스키마)
Redis 8 (Refresh Token)
Apache Kafka 4.3 (KRaft)
Flyway (스키마 버전 관리)

External Integration

Google Gemini API
Slack API
(DM · Interactivity)
Health Data Source
(Health Connect 등)

Infra · DevOps

Docker · Docker Compose
Nginx (Blue-Green 전환)
GitHub Actions CI/CD AWS
배포 환경

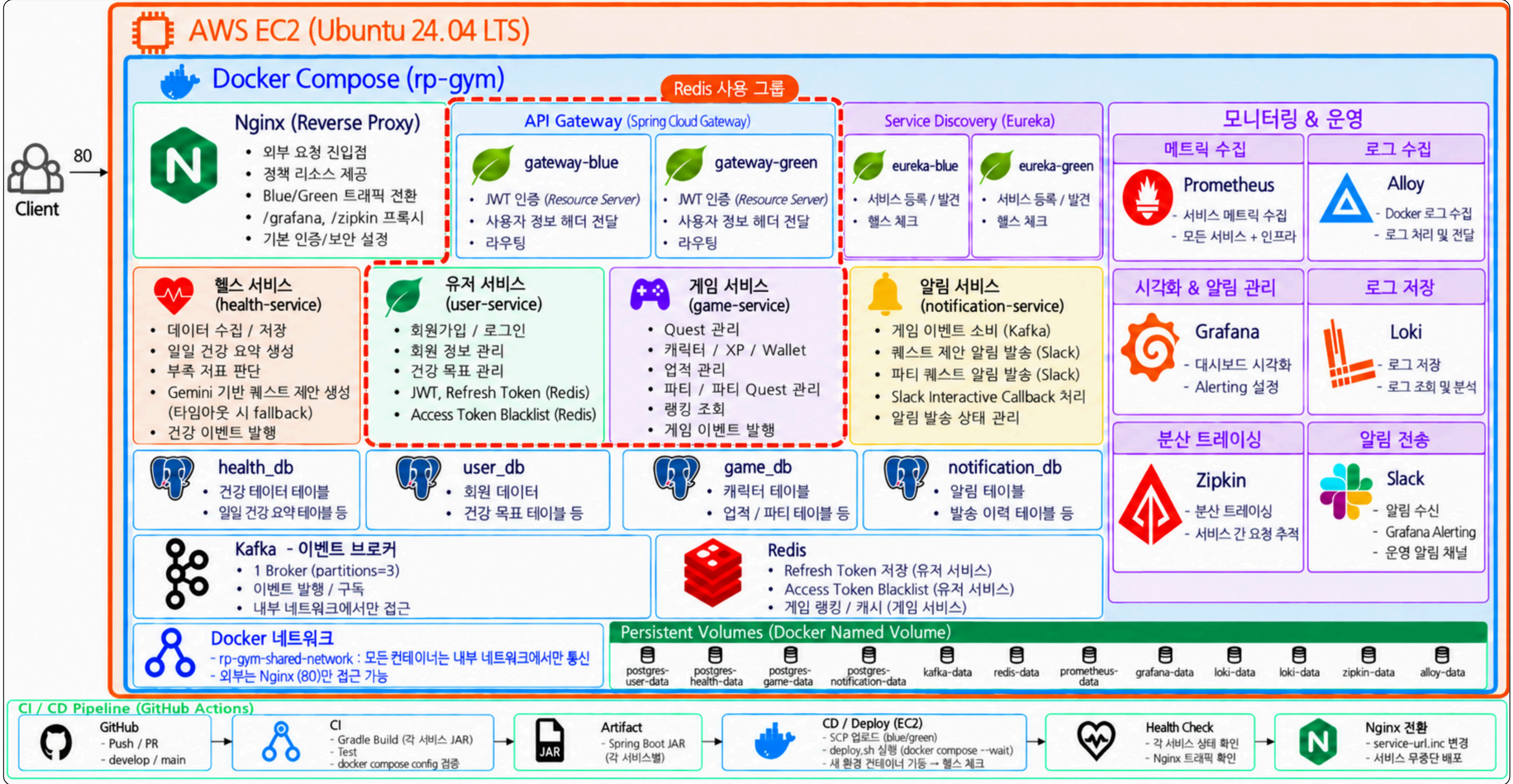
Observability

Prometheus · Grafana Loki ·
Grafana Alloy Zipkin ·
Micrometer Tracing
Springdoc Swagger

Test · Collaboration

JUnit 5 · Mockito · AssertJ
Testcontainers · Awaitility
Postman · JMeter GitHub
(Issue · PR) · Notion · Slack

프로젝트 구조(인프라 설계도)



프로젝트 팀 구성 및 역할

ROLE & SERVICE OWNERSHIP

공통 기반 · 인프라 · 배포	User · Notification	Health Service	Game Service
정나영 팀장 · 개발 리드	황지호 팀원	진혜림 팀원	최유준 팀원
담당 업무 - 개발 리드(서비스 설계 및 개발 방향 조율) - 인프라·CI/CD·배포 - 모니터링 환경 구축	담당 업무 - 사용자 인증·바디 프로필·일일 건강 목표 - 일일·Party Quest Slack 알림	담당 업무 - 건강 활동 데이터 수집·분석 - AI Quest 제안 이벤트 발행	담당 업무 - Quest·Character·Achievement·XP - Party·Ranking

프로젝트 수행 절차 및 방법

구분	기간	활동	비고
사전 기획	8/25(화) ~ 8/31(월)	서비스 기획 및 MSA·인프라 구조 설계	아이디어 선정
MVP 핵심 경로 구현	9/1(화) ~ 9/7(월)	회원·인증·건강 활동 등 핵심 기능 및 서비스 연동 구현	MVP 중간 발표회
AI Quest·보상·배포 연동	9/8(화) ~ 9/14(월)	AI Quest·보상 기능 구현 및 배포 환경 구축	
안정성·동시성 개선	9/15(화) ~ 9/21(월)	인증·Gateway 개선 및 모니터링 환경 구축	최적화, 오류 수정
트러블슈팅·최종 고도화	9/22(화) ~ 9/28(월)	추가 기능 개발 및 테스트·배포 안정화	
총 개발 기간	8/25(화) ~ 9/28(월) 총 5주		

1주차

건강 활동을 Quest·보상으로 연결하는 **서비스 기획과 핵심 흐름을 확정**하고, MSA 4개 서비스 + Gateway·Eureka 구조와 AWS EC2·Docker Compose 기반 **인프라를 설계**한 뒤 **역할 분담·그룹 리포지토리 생성·이슈/PR 템플릿까지 개발 준비를 마칩**

2주차

기능을 이슈 단위로 쪼개 브랜치 작업을 시작해 공통 모듈·Eureka·서비스 모듈과 Docker Compose·CI 골격을 올리고, 회원가입·로그인(JWT)·Gateway 인증 라우팅·캐릭터 조회·건강 활동 동기화와 Outbox→Kafka 발행까지 **MVP 핵심 경로를 구현**함

3주차

AI(Gemini) Quest 제안, Quest 수행·XP·랭킹, 바디 프로필·일일 목표, Blue-Green 배포·Flyway 연동을 붙여 9/10에 **MVP 개발을 마무리**하고, 운영 배포와 통합테스트 과정에서 나온 Gemini 타임아웃·Blue-Green 배포 장애 시 롤백 미지원·아웃박스 발행 순서 문제를 **트러블 슈팅**함

4주차

튜터 피드백을 반영해 토큰 재발급(RTR)·로그아웃·회원 탈퇴로 **인증 흐름을 완성**하고, Gateway 보안·요청 처리를 **리팩터링했으며**, Prometheus·Grafana·Loki·Zipkin 모니터링 환경과 아웃박스 정리 배치·지표, Notification **서비스 모듈을 새로 구축**함

5주차

파티·파티 퀘스트·업적, Slack Quest DM과 수락/거절 처리, Health Connect 외부 수집 경로, Gateway 통합 Swagger까지 추가 기능을 완성하고 전체 E2E **통합테스트와 부하테스트를 정리해 최종 보고서를 작성**함

ERD (핵심 테이블)

User Service

users
id · email · slack_id



user_daily_health_goals
일일 활동 목표
(걸음 수, 활동 시간, 활동 칼로리)

Health Service

health_activities
30분 주기 누적 활동량



daily_health_summaries
일자별 합계 · 달성 여부



event_outbox
Kafka 발행 대기 이벤트

Game Service

quest_suggestions
AI 제안 · 30분 유효



quests
baseline · status

xp_ledgers
XP 지급 원장



wallets
user_id · 누적 xp



characters
user_id · level

Notification

quest_offers
suggestion_id(UK) ·
Slack 발송 상태

API 명세 (핵심 API)

구분	Method	Endpoint	기능
인증	POST	/api/v1/users/login · /refresh	JWT 발급 · 토큰 회전(RTR)
일일 목표	POST · GET	/api/v1/daily-goals/me	일일 건강 목표(걸음 수·활동 시간·활동 칼로리)
건강	POST	/api/v1/health-activities/sync	누적 활동량 동기화 → 흐름 시작
건강	GET	/api/v1/health-activities/today	오늘의 건강 활동 조회
Slack	POST	/api/v1/notifications/slack/interactions	Slack에서 Quest 수락·거절 버튼 콜백
내부	POST	/api/v2/internal/games/quest-suggestions/{id}/accept	제안 수락 → Quest 생성
퀘스트	GET	/api/v1/games/quests/me/active	진행 중인 Quest·진행도 조회
보상	GET	/api/v1/games/characters/me · /rankings	레벨 · XP · 랭킹 조회
파티	POST	/api/v1/games/parties · /party-quests	파티 생성 · 파티 Quest 시작

외부 API

 Gateway가 JWT 검증 후 X-UserId · X-UserRole 헤더 전달

내부 API

 /api/v1·2/internal/** 는 외부 요청 차단, 서비스 간 Feign 전용

활용 기술 및 구현 방법

Health · Game

Kafka + Outbox

이벤트를 DB와 함께 커밋한 뒤 발행해 유실 없이 순서 보장

Health Service

Gemini API

가장 부족한 지표로 맞춤 Quest 생성, 실패하면 기본 Quest

Notification

Slack Bot API

Quest 카드를 DM으로 발송, HMAC 서명으로 위조 요청 차단

User Service

Redis + Lua Script

Refresh Token 회전을 원자 실행, 동시 재발급은 1건만 성공

Game Service

Redis ZSET

XP 지급 직후 랭킹 갱신, 동점자는 같은 순위로 계산

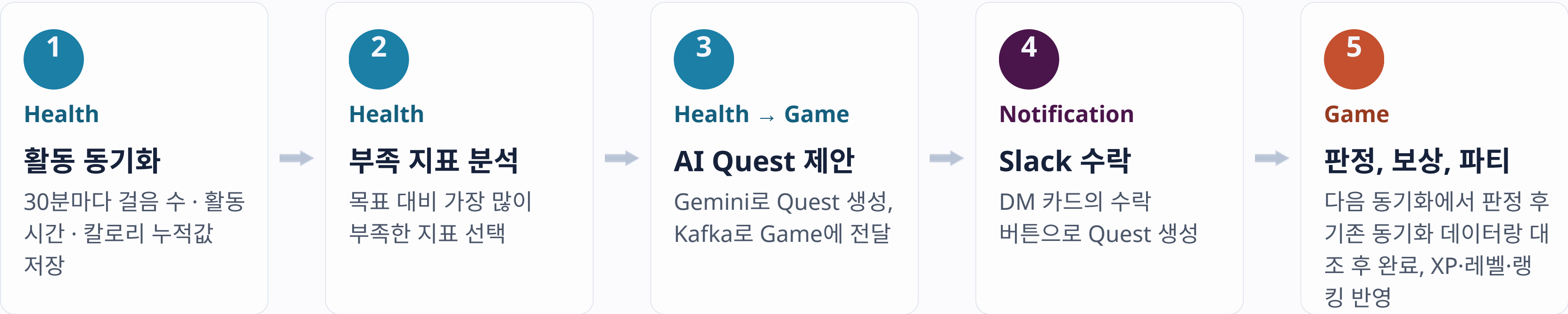
Gateway

Spring Cloud Gateway

JWT 검증 후 X-UserId, X-UserRole 헤더 전달, 내부 API는 외부 차단

핵심 기능 흐름

건강 데이터 동기화가 맞춤 Quest와 게임 보상으로 이어지는 과정



예시 목표 5,000보 현재 3,200보 부족 1,800보

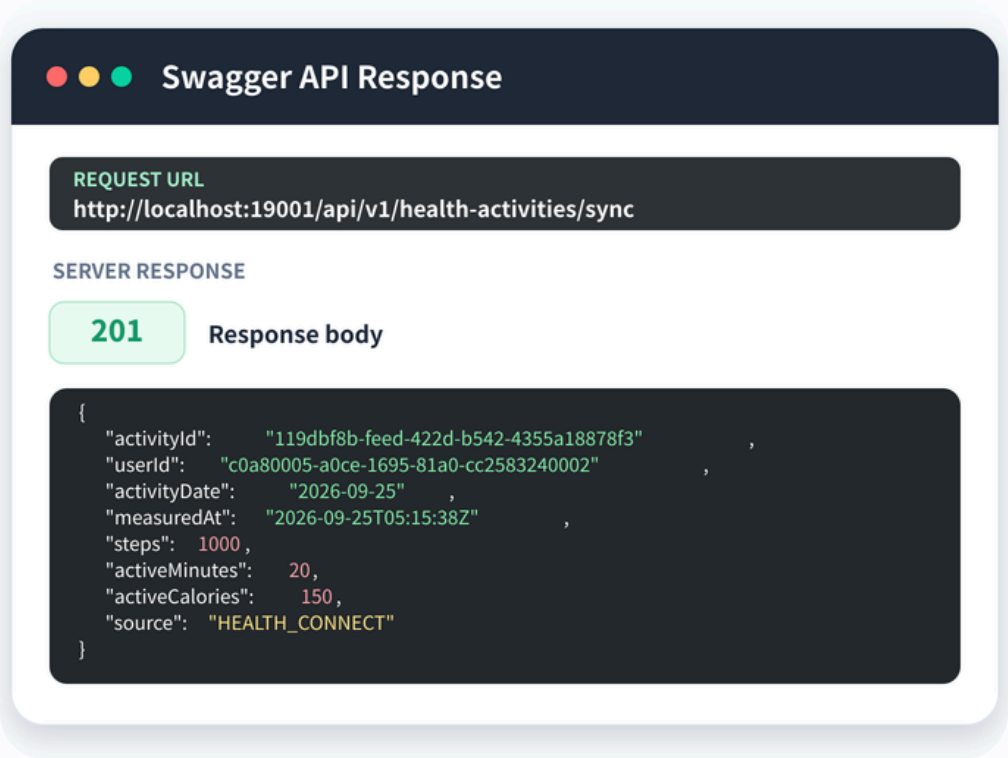
Quest · 1,500보 더 걸기

수락 시점의 3,200보를 기준값으로 고정 → 누적 4,700보가 동기화되면 자동 완료 · +5 XP

서비스 간 연결 User → Feign → Health → Kafka health.events → Game → Kafka game.events → Notification → 수락 HTTP → Game

구현 결과

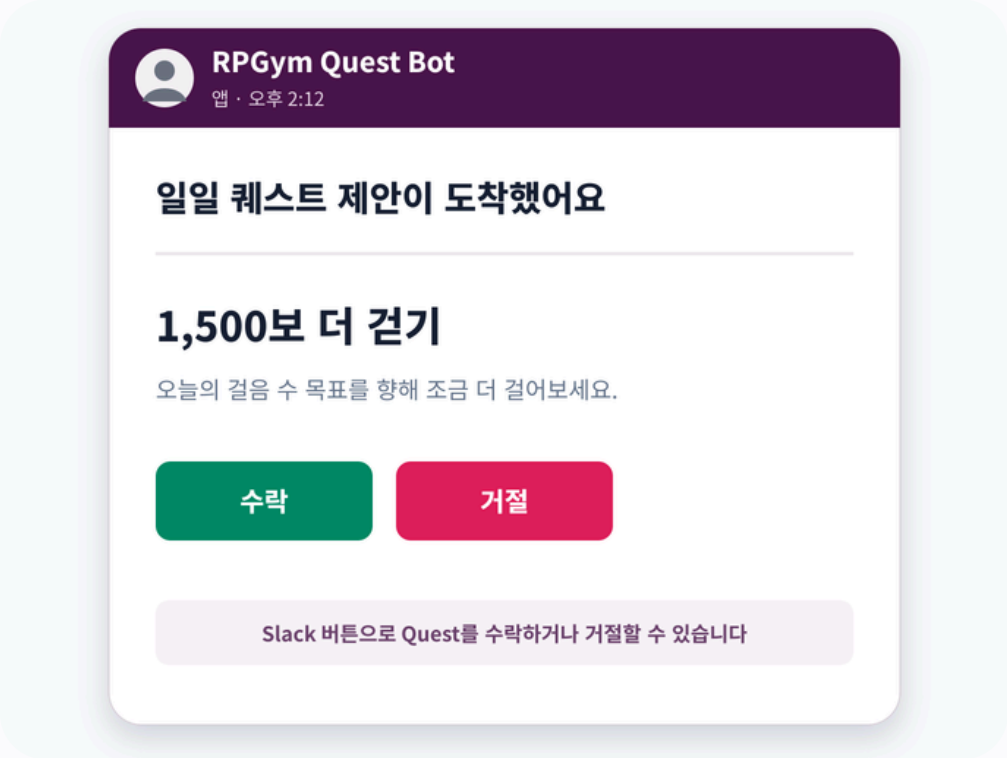
1 건강 데이터 동기화



[Swagger]

POST /health-activities/sync

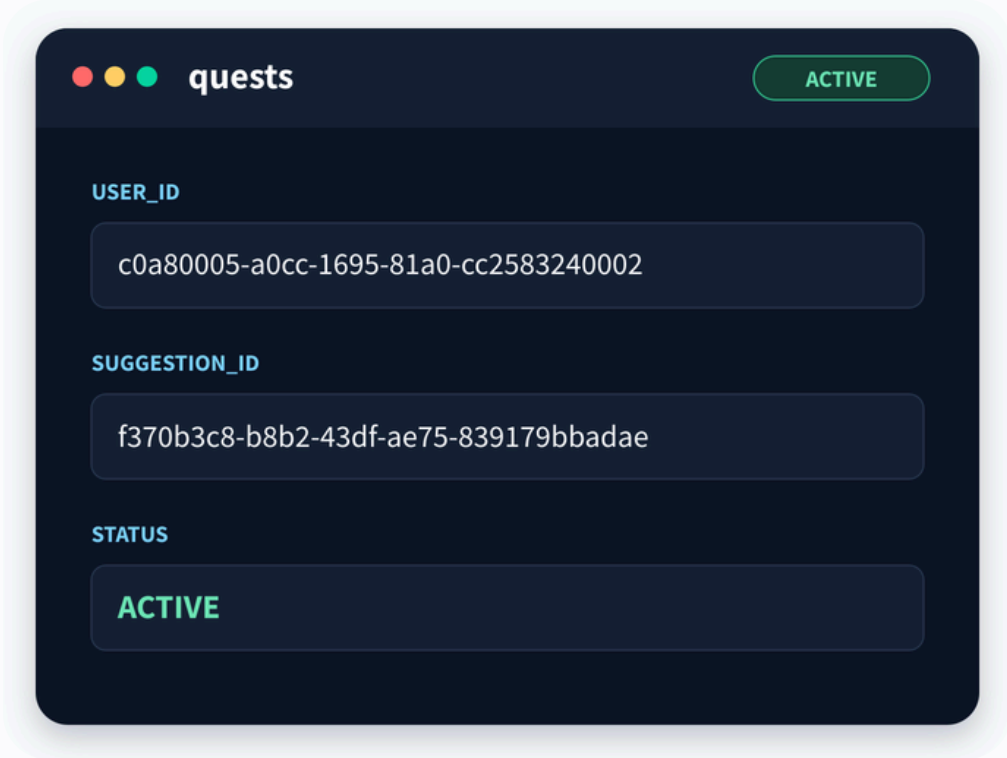
2 Slack Quest 알림



[Slack DM]

RPGym Bot · 수락 / 거절 버튼

3 DB에 생성된 Quest



[DB 조회]

game_service.quests · status ACTIVE

기획·설계

서비스 컨셉을 도메인 경계와 핵심 기능으로 구체화

01

서비스 경험 설계

목표 설정 → 부족 지표 분석 →
Quest 제안 → 수행·보상으로 연결

02

MSA 경계 설정

User는 회원·목표, Health는 활동
데이터, Game은 Quest·파티,
Notification은 알림을 담당

03

서비스 탐색과 인증 처리

Eureka로 서비스를 탐색하고
Gateway에서 JWT를 검증해 사용자
식별 정보를 전달

데이터 소유권을 기준으로 서비스를 나누고, Gateway와 Eureka로 요청 흐름을 구성

서비스 연동과 안정성 확보

서비스 간 통신 방식과 데이터 정합성을 실제 흐름에 적용

01

핵심 요청 처리

건강 데이터 동기화 후 목표 대비
부족 지표를 계산하고 개인 맞춤
Quest를 제안

02

REST와 Kafka로 서비스 연결

즉시 필요한 조회는 REST/Feign,
Quest·알림 후처리는 Kafka 이벤트로
분리

03

이벤트 누락·중복과 AI 장애 대응

Outbox로 데이터 변경과 이벤트
기록을 묶고, 중복 처리 방지 및 AI
실패 시 기본 Quest 적용

즉시 필요한 조회는 동기 호출로, Quest와 알림 후처리는 Kafka 이벤트로 분리

배포·운영

Docker로 실행 환경을 통일하고, 배포와 모니터링 체계를 구축

01

환경과 배포

Docker Compose로 DB·Kafka·Redis를 구성하고 GitHub Actions로 빌드·테스트·배포

02

전환과 복구

Blue/Green 환경의 Health Check를 통과한 뒤 Nginx 트래픽을 전환하고 실패 시 롤백

03

관측과 개선

Prometheus·Grafana로 메트릭, Loki로 로그, Zipkin으로 서비스 간 요청을 확인

Docker로 실행 환경을 맞추고, 배포 후 메트릭·로그·추적으로 서비스 상태를 확인

서비스는 어디서 막힐까?

30분 마다 몰리는 이벤트를 한꺼번에 쌓아 처리 속도와 CPU 사용량 확인했다.

01 정각 이벤트 집중을 가정

30분마다 전 유저의 건강 데이터가 동기화되고
Kafka 이벤트가 게임 서비스로 한꺼번에 유입된다.
정각에 이벤트가 집중되는 상황을 최악의 부하로 가정했다.

02 이벤트를 쌓고 처리 시간 측정

Jmeter로 kafka 이벤트를 몰아 발행
게임 서비스를 잠시 멈춰 큐에 적재
재개 후 전체 처리 완료까지 측정

03 경로 별 처리량과 30분 처리 가능량(20,000건)

처리 경로	처리량	건당시간	30분 처리량
가벼운 경로	800건/s	3.7ms	약144만명
무거운 경로	423건/s	7.1ms	약 76만명

가벼운 경로: 진행중인 퀘스트가 없는 유저.

무거운 경로: 개인 퀘스트와 4인 파티퀘스트가 둘다 진행중인 유저

04 병목 발견

처리량은 절반으로 줄었지만 cpu는 약 0.7코어로 비슷했다. 계산보
다 대기가 길다는 단서를 발견

DB 커밋 대기가 처리 시간을 좌우했다

스레드와 DB세션을 함께 관찰해 이벤트 처리시간이 어디서 소비되는지 추적

01 처리 스레드의 70% 이상이 DB 대기

쿼리 응답 대기 43%
커밋 완료 대기 30%
코드 실행 17%
kafka offset 기록 7%

02 DB 세션의 58.5%가 WAL 기록 대기

WAL 디스크 기록 완료를 기다렸지만 디스크 읽기와 행 잠금 대기는 관찰되지 않아서 조희, 경합이 주 원인은 아니었다.

03 커밋 횟수와 처리시간이 함께 증가

가벼운 경로는 WAL 기록이 필요한 커밋 1회에 3.7ms, 무거운 경로는 건당 2.07회에 7.1ms.
즉 커밋마다 발생하는 WAL 대기가 핵심 병목

04 결론

진행 중인 퀘스트를 갱신하는 무거운 경로 기준으로, 측정한 처리량(초당 423건)을 30분으로 환산하면 약 76만 명까지 소화할 수 있습니다. 그 이상이 한꺼번에 몰리면 다음 주기 전에 다 처리하지 못해 lag이 쌓입니다.

병목은 커밋할 때마다 기다리는 WAL 디스크 기록이므로, 개인·파티 처리를 한 트랜잭션으로 묶어 커밋을 2회에서 1회로 줄이는 것이 1순위 개선안이지만 현재 서비스는 데모 프로젝트이고 서비스 확장이 뒷날 개선해야 할 상황이라고 판단해서 제시한 가설을 실제로 옮기지 않았습니다.

05 한계

76만 명이란 수치는 초당 423건이 규모를 키워도 그대로 유지된다고 가정하고 30분(1,800초)을 곱한 값입니다(실제론 20,000). 실제로 76만 건을 넣어본 적은 없습니다.

규모가 커지면 이 가정이 깨질 수 있는 요인들이 존재하는데 이를 추정만 했을뿐 실제 실험해보진 못했습니다.

파티 퀘스트 동시 완료 시 XP 중복 지급 차단

서로 다른 파티션에서 들어오는 동시 갱신을 공유 카운터와 조건부 UPDATE 선점으로 해결

01 문제 상황

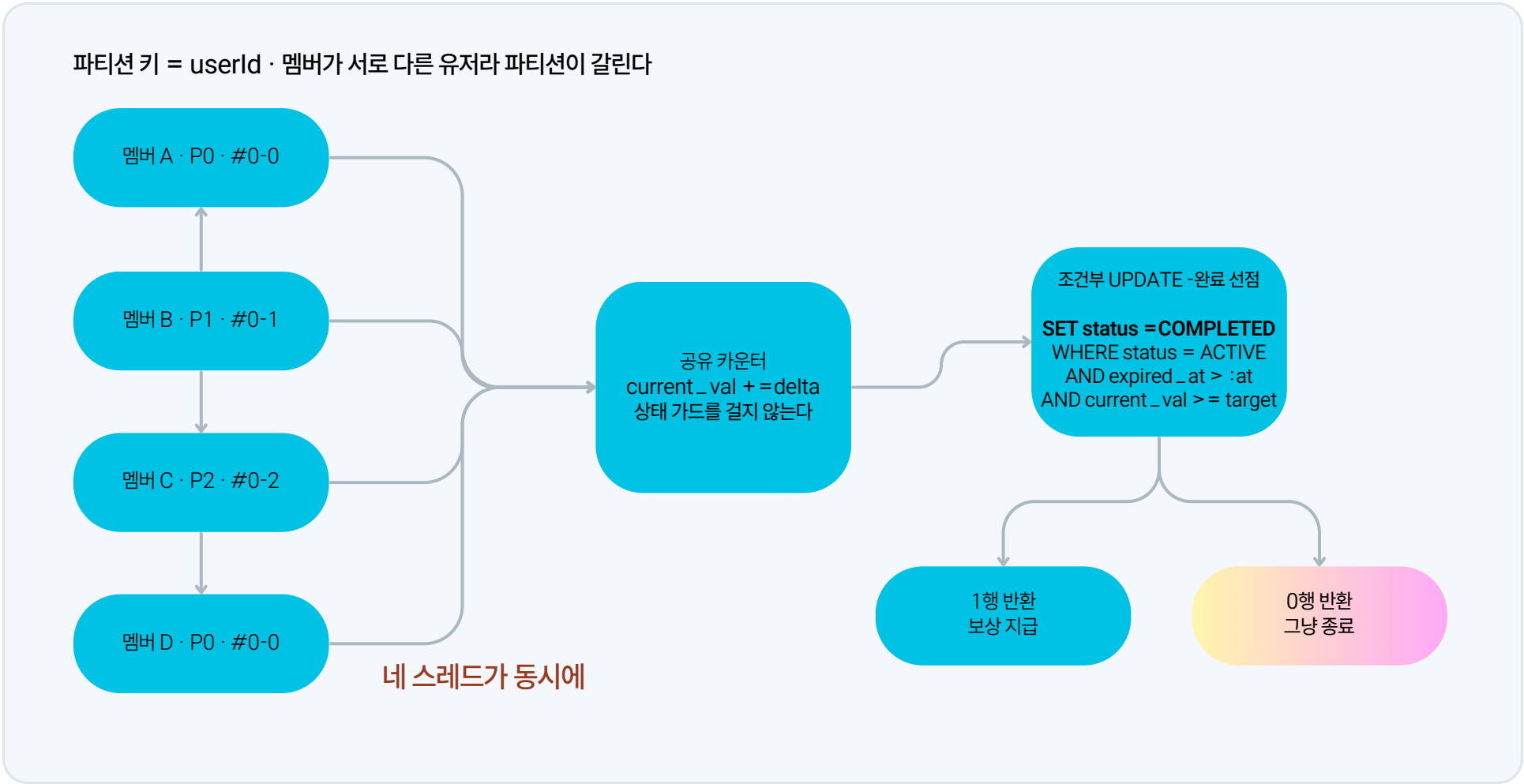
파티 멤버는 서로 다른 유저라 이벤트가 다른 파티션·스레드에
서 같은 행을 동시에 갱신합니다. 모두가 완료로 판단하면 XP가
4배로 지급되고, 각자 합계를 읽으면 커밋 전 기여가 보이지 않
아 완료가 누락됩니다.

02 트레이드오프

기여분 즉시 합산은 완료 누락이, 비관적 락은 파티가 늘수록 처
리량 병목이 남습니다. 두 실패 모드의 처방이 반대라 하나의 장
치로는 둘 다 막을 수 없었습니다.

03 최종 결정 및 근거

공유 카운터는 가드 없이 누적만 하고, 상태·만료·목표 조건을
하나의 조건부 UPDATE에 묶었습니다. 1행을 반환받은 트랜잭
션만 같은 트랜잭션 안에서 보상을 지급합니다.



핵심 결과	XP 중복 지급 0건	완료 누락 0건
파티 100개 × 4명 동시 주입 · 3스레드 실경쟁에서 XP 원장 정확히 400건, 정합성 위반 0행		

Refresh Token Rotation 원자성 보장

개별 Redis 명령의 부분 실패와 동시 요청 문제를 Lua Script로 해결

01 문제 상황

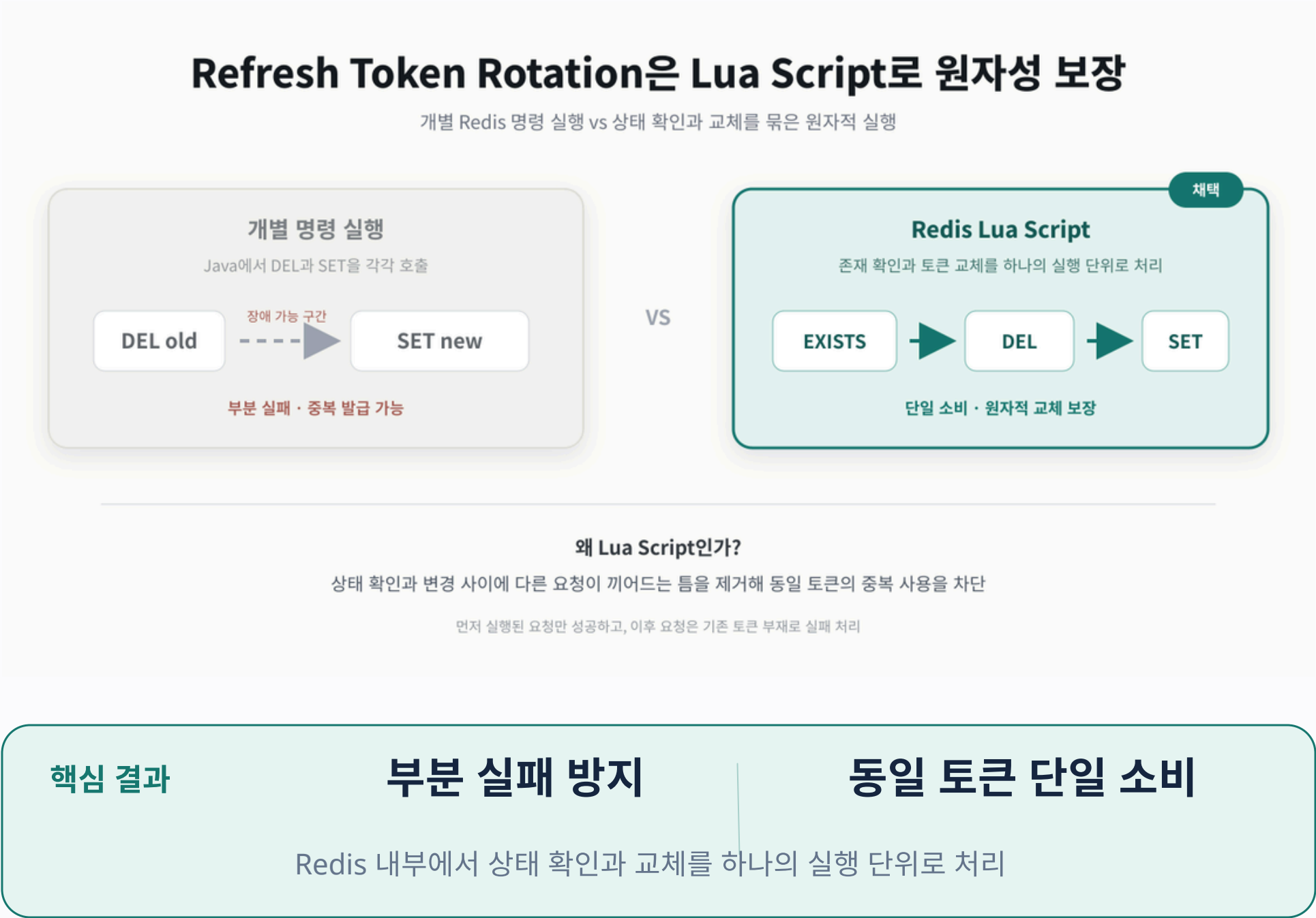
기존 토큰 삭제(DEL)와 새 토큰 저장(SET)을 개별 실행하면, 중간 장애로 새 토큰이 저장되지 않거나 동시 요청에서 여러 토큰이 발급될 수 있었습니다.

02 트레이드오프

개별 DEL·SET은 구현이 단순하지만 부분 실패와 경쟁 조건이 남습니다. Lua Script는 구현이 복잡해지는 대신 원자성과 단일 소비를 보장합니다.

03 최종 결정 및 근거

EXISTS·DEL·SET을 하나의 Lua Script로 묶었습니다. 기존 토큰이 남아 있을 때만 교체하므로 동일 토큰의 동시 요청 중 하나만 성공합니다.



Health&Game 이벤트 순서 보장을 위한 단일 토픽 설계

Synced · QuestSuggested 순서 역전으로 Quest baseline이 들어지는 문제를 단일 토픽 + userId 파티션 키로 해결

01 문제 상황

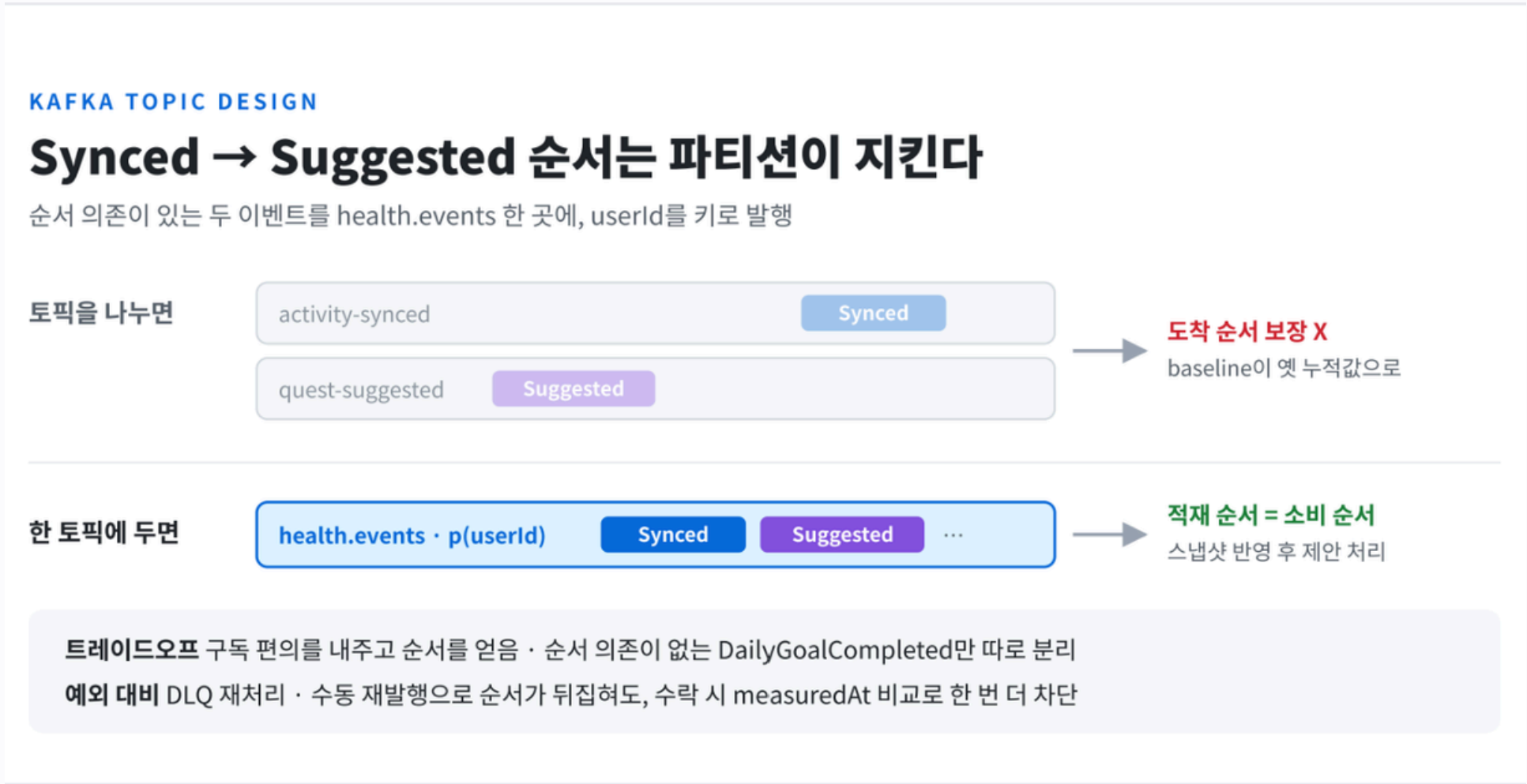
한 번의 동기화에서 Synced와 QuestSuggested가 함께 발행됩니다. 토픽을 분리하면 두 이벤트가 독립적으로 소비되어 순서가 뒤집힐 수 있고, Game Service가 Quest baseline을 옛 누적값으로 고정해 진행도 계산이 깨질 수 있었습니다.

02 트레이드오프

타입별 토픽 분리는 구독이 편하지만 토픽 간 순서를 보장하지 못합니다. 단일 토픽은 컨슈머가 eventType으로 분기해야 하는 대신, 같은 사용자 이벤트의 처리 순서를 보장합니다.

03 최종 결정 및 근거

두 이벤트를 health.events 단일 토픽에 userId 키로 발행했습니다. 순서 의존이 없는 DailyGoalCompleted만 분리하고, DLQ 재처리 같은 예외는 수락 시 measuredAt 비교로 한 번 더 차단합니다.



핵심 결과

순서 역전 방지

baseline 정합성 보장

같은 사용자의 이벤트는 같은 파티션에서 발행 순서대로 소비

Gemini 호출 트랜잭션 경계 분리

외부 API 지연으로 인한 DB 커넥션 점유 위험을 트랜잭션 경계 분리로 해결

01 문제 상황

Quest 제목 생성을 위한 Gemini API 호출이 DB 트랜잭션 내부에서 동기로 실행되고 있었음.
제안 주기가 30분마다 바뀌며 빈도가 48배 늘었고,
요청 1건이 최대 14.5초까지 DB 커넥션을 점유할 수 있었음.

02 트레이드오프

트랜잭션 내부 동기 호출은 구현이 단순하지만,
외부 API 지연만큼 DB 커넥션을 점유함.
Kafka 비동기 처리는 트랜잭션과 완전히 분리되지만,
같은 서비스 내부 처리 치고는 구조가 과함.

03 최종 결정 및 근거

AFTER_COMMIT + @Async 방식을 채택함.
같은 서비스 내부 후속 처리에 Kafka 까지 쓰는 건
구조를 과하게 키운다고 판단함.
이후 리뷰를 거치며 전파 충돌, 이벤트 유실, 배포 타이밍 문제를
순차적으로 보완함.



트랜잭션 내부 동기 호출

+ 구현이 단순함
- 외부 API 지연만큼 DB 커넥션을 점유

Kafka 비동기 처리

+ 트랜잭션과 완전히 분리됨
- 같은 서비스 내부 처리치곤 구조가 과함

AFTER_COMMIT + @Async

+ 분리되면서도 구조는 단순하게 유지
+ 이후 리뷰로 안정성까지 순차 보완

조건부 UPDATE 하나로 파티 정원 보장

읽고 검사한 뒤 +1하는 두 단계 사이의 틈을 DB 한 문장으로 없앴

01 문제 상황

엔티티를 읽어 빈자리를 확인한 뒤 currentMember + 1을 하면, 두 요청이 동시에 "빈자리 있음"을 보고 통과할 수 있습니다. 마지막 한 자리에 두 명이 동시에 들어오면 5명이 되고, 파티 퀘스트 완료 시 XP도 5번 지급됨.

02 트레이드오프

비관적 락은 확실하지만, 인기 있는 파티일수록 같은 행에서 대기 길어집니다. 조건부 UPDATE는 여러 단계 로직을 담을 수 없지만, 검사와 증가를 한 문장에서 끝낼 수 있음.

03 최종 결정 및 근거

모집 상태 · 마감 시각 · 빈자리 조건을 WHERE에 넣고, 같은 문장에서 +1을 합니다 (reserveSeat). 영향받은 행이 1이면 성공, 0이면 정원 초과로 처리하고, DB CHECK 제약을 마지막 방어선으로 둡니다.



Blue-Green 배포 안정성 보장

신규 환경 검증부터 트래픽 전환, 장애 시 자동 롤백까지 배포 흐름 개선

01 문제 상황

Blue-Green 전환만으로 전체 배포의 안정성을 보장하기 어려웠습니다. main 연속 반영 시 동시 배포 충돌 가능성이 있었고, .env·Secret 누락이나 Compose 설정 오류를 배포 단계에서야 발견할 수 있었습니다.

또한 최초 배포와 재배포는 복구 가능한 이전 환경의 존재 여부가 달랐습니다.

02 트레이드오프

단순한 배포 스크립트는 관리하기 쉽지만 오류를 운영 서버 반영 이후 발견할 위험이 있습니다.

사전 검증·동시 실행 제어·자동 롤백을 추가하면 구조는 복잡해 지지만, 운영 영향과 배포 상태 충돌을 줄이는 안정성을 우선했습니다.

03 최종 결정 및 근거

CI에서 docker compose config -q로 설정을 사전 검증하고, GitHub Secrets 기반으로 EC2 .env를 자동 생성했습니다.

CI는 이전 실행을 취소해 최신 변경을 빠르게 검증하고, CD는 순차 실행하도록 concurrency를 분리했습니다.

EC2에서도 flock으로 중복 배포를 차단하고, 최초/재배포 상태를 구분해 실패 시 자동 롤백하도록 구성했습니다.

Blue-Green 배포 자동화 흐름 개선

배포 오류를 사전에 차단하고, 순차 실행과 자동 복구까지 보장



핵심 결과 배포 오류 사전 차단 중복 배포 방지 장애 시 자동 복구

Redis 내부에서 상태 확인과 교체를 하나의 실행 단위로 처리

AI 설계 검증 로그

검증 항목	Kafka 토픽 분리 전략	파티셔닝 전략	이벤트 발행 시점과 원자성
프롬프트	"Health Service가HealthActivitySynced, QuestSuggested, DailyGoalCompleted 세 이벤트를 발행하는데, 지금 health.events 단일 토픽으로 보내고 있어. 이벤트 타입별로 토픽을 나누는 게 맞을지 검토해줘."	파티션 수와 키는 어떻게 잡아야 순서 보장과 병렬성을 동시에 얻을 수 있을 지 검토해줘."	"HealthActivityService.sync() 에서 활동을 저장한 뒤 KafkaTemplate 으로 바로 발행하려 했는데, DB 커밋과 Kafka 발행 사이에 장애가 나면 어떻게 되는지 검토해줘."
AI 답변	"이벤트 타입별 토픽 분리를 권장. 컨슈머가 관심 있는 이벤트만 구독할 수 있고, 토픽별로 보존 정책·파티션 수를 다르게 가져갈 수 있음."	"파티션 키를 userId로 두면 사용자 단위 순서는 보장되면서 사용자 간에는 병렬 처리가 가능. 파티션 수는 컨슈머 인스턴스 수 이상으로."	"DB 트랜잭션과 Kafka 발행은 별개라 원자성이 없음. 저장은 됐는데 발행이 실패하면 이벤트 유실. Transactional Outbox 패턴 권장."
수용 여부	✗ 거부	✓ 수용	✓ 수용
판단 근거	한 번의 동기화에서 HealthActivitySynced 와 QuestSuggested가 함께 발행된다. 토픽이 분리되면 두 이벤트가 독립적으로 소비되어 순서가 뒤집힐 수 있고, 그 경우 Game Service가 Quest baseline을 옛 누적값으로 고정해 진행도 계산이 깨진다.	순서 보장이 필요한 범위가 "전체"가 아니라 " 한 사용자 안에서 " 다. 사용자 A와 B의 이벤트 순서는 서로 무관하므로, userId를 키로 두면 필요한 순서만 지키면서 병렬성을 얻는다. 파티션은 3개로 두어 컨슈머를 3대까지 확장할 여지를 남겼다.	같은 트랜잭션에서 event_outbox 에 적재하고, 별도 스케줄러가 PENDING을 폴링해 발행하도록 분리했다. 발행 실패는 재시도되고 max-retry 초과 시 DLQ로 보낸다. 건강 데이터 저장은 즉시 완료되고 후속 처리만 지연되므로 사용자 응답 속도에도 영향이 없다.

자체 평가 의견

10점 만점

팀원 6명이 프로젝트 완성도와 개선점을 각자의 관점에서 평가

정나영

7 / 10

잘한 점 - Gateway 인증·인가와 Blue-Green 배포 환경을 구성하고, Health Check·자동 롤백·중복 배포 방지까지 적용해 기능 구현을 넘어 실제 운영 안정성을 고려한 배포 구조를 설계하고 개선했습니다.

아쉬운 점 - 다양한 의견을 조율하는 과정에서 리더로서 명확한 기준을 제시하고 결정을 이끄는 역할이 부족했습니다. 다음에는 의견을 충분히 수렴하되 필요한 순간에는 방향을 명확히 제시하는 리더십을 갖추고 싶습니다.

황지호

7 / 10

잘한 점 - 처음 접한 Kafka 등의 기술도 직접 학습하고 구현 방식의 트레이드오프를 고민하며 프로젝트에 적용함으로써, 새로운 기술에 도전하고 문제를 해결해 나갔습니다.

아쉬운 점 - 개발 기간이 부족해 일부 기능을 TODO로 남겼으며, 다양한 예외 상황과 장애 시나리오를 충분히 구현하고 검증하지 못한 점이 아쉽습니다.

진혜림

8 / 10

잘한 점 - 건강 데이터를 '당일 누적 스냅샷'으로 모델링하고, (userId, measuredAt) 기준으로 역등하게 처리하며 Transactional Outbox로 이벤트를 발행해, 수집 채널이 달라도 중복 요청이나 재전송이 와도 데이터가 어긋나지 않는 동기화 파이프라인을 설계했습니다.

아쉬운 점 - 개발 기간이 부족해 성능 향상과 부하 테스트 수치로 개선 효과를 증명하지 못해 아쉽습니다.

최유준

4 / 10

잘한 점 - 내 서비스에 적합한 설계가 무엇이고 어떤 트레이드오프가 있겠는지 계속 고민해보고 설계한대로 흐름이 흘러갈때의 재미를 느꼈습니다.

아쉬운 점 - DB 엔진 스프링 트랜잭션 롤백 핸들러 등 실질적으로 코딩할 때 고려 못한점도 많고 그걸 구현하지 못하고 AI한테 맡겼던 점이 루키가 가져야 할 핵심 역량을 못 챙긴 것 같아서 아쉬웠습니다.

강윤석

4 / 10

잘한 점 - Gemini 호출을 트랜잭션 밖으로 분리하면서 동기 호출, Kafka 비동기, AFTER_COMMIT + @Async까지 트레이드오프를 비교해 구조를 과하게 키우지 않는 선택을 한 것이 기억에 남습니다.

아쉬운 점 - 트랜잭션 전파 충돌, 배포 시 컨테이너 종료 대기시간 등 문제를 미리 점검하지 못하고 뒤늦게 발견한 점. 다음에는 실제 배포됐을 때의 흐름까지 설계해보고 싶습니다.

남건우

5 / 10

잘한 점 - 제가 담당한 서비스의 도메인에서 랭킹 구현에서 Redis를 사용하며 적절히 설계해 구현했다고 생각해 뿌듯합니다.

아쉬운 점 - 작업량과 각종 버그로인해 기간이 부족해 팀원들의 코드를 리뷰를 별로 하지 못한것이 아쉽습니다.

시연 영상(핵심 기능)

일일 건강 목표 등록

Gemini에게 물어보기

ex.html?url.primaryName=User+Service

☆

↓

na

⋮

구직

ai

단기심화

개발

NAVER

daily-health-goal-controller

GET

/api/v1/daily-goals/me

🔒

⌵

POST

/api/v1/daily-goals/me

🔒

⬆

Parameters

No parameters

Cancel

Reset

Request body required

application/json

⌵

Edit Value

Schema

```
{  "stepGoal": 5000,  "activeMinutesGoal": 180,  "activeCaloriesGoal": 300}
```

시연 영상(핵심 기능)

Swagger UI

43.200.62.125/swagger-ui/index.html?urls.primaryName=Game+Service#/party-controller/create_1

party-controller

POST /api/v1/games/parties

Parameters

No parameters

Request body required

application/json

Edit Value Schema

```
{  "partyName": "walking",  "visibility": "PUBLIC",  "metric": "STEPS"}
```

Execute

파티 생성부터 퀘스트 알림까지 API 검증

25°C
부분적으로 맑음

오후 3:12
2026-09-28

감사합니다

